



DECEMBER 12, 2025

HACKING WEB APPLICATIONS

MODULE 14

VYANKATESH ANDIL



TABLE OF CONTENTS

Contents

1: Introduction to Web Application Security	1
1.1 Introduction	1
1.2 What is a Web Application?	1
1.3 Web Application Architecture Overview	1
1.3.1 Client Side	1
1.3.2 Server Side	2
1.3.3 Database Layer	2
1.4 Why Web Applications Are Targeted	2
1.5 Common Security Challenges in Web Applications	2
1.6 Difference Between Network and Web Application Attacks	3
1.7 OWASP and Web Application Security	3
1.8 Impact of Web Application Vulnerabilities	3
1.9 Ethical Hacking Perspective	4
1.10 Legal and Compliance Considerations	4
1.11 Scope of Module 14	4
1.12 Learning Objectives	5
1.13 Summary of Part 1	5
2: Web Application Architecture and Attack Surface	6
2.1 Introduction to Web Application Architecture	6
2.2 Three-Tier Web Application Architecture	6
2.2.1 Presentation Layer (Client Layer)	6
2.2.2 Application Layer (Server Layer)	6
2.2.3 Data Layer (Database Layer)	7
2.3 Supporting Components in Web Applications	7
2.4 Data Flow in Web Applications	8
2.5 Trust Boundaries in Web Applications	8
2.6 What is an Attack Surface?	8
2.7 Client-Side Attack Surface	9

TABLE OF CONTENTS

2.8 Server-Side Attack Surface	9
2.9 API Attack Surface.....	10
2.10 Third-Party and Dependency Risks	10
2.11 Configuration-Based Attack Surface.....	10
2.12 Importance of Architectural Understanding for Ethical Hackers	11
3: Common Web Application Vulnerabilities	12
3.1 Introduction to Web Application Vulnerabilities	12
3.2 OWASP Top 10 and CEH Alignment	12
3.3 Injection Vulnerabilities	12
3.4 Broken Authentication.....	13
3.5 Broken Access Control	13
3.6 Security Misconfiguration	14
3.7 Sensitive Data Exposure	14
3.8 Cross-Site Scripting (XSS).....	15
3.9 Insecure Deserialization	15
3.10 Using Components with Known Vulnerabilities	15
3.11 Insufficient Logging and Monitoring	16
3.12 Business Logic Flaws	16
3.13 Client-Side Security Weaknesses	16
3.14 Impact of Combined Vulnerabilities	17
3.15 Ethical Hacking Perspective.....	17
4: Detection and Assessment of Web Application Vulnerabilities	18
4.1 Introduction to Web Application Vulnerability Detection	18
4.2 Challenges in Web Application Vulnerability Detection	18
4.3 Web Application Reconnaissance	18
4.4 Input Validation and Data Flow Analysis.....	19
4.5 Authentication and Authorization Testing.....	19
4.6 Session Management Analysis	19
4.7 Error Handling and Exception Analysis.....	20
4.8 Business Logic Assessment	20

TABLE OF CONTENTS

4.9 Client-Side Code Review	20
4.10 API Security Assessment	20
4.11 Configuration and Deployment Review	21
4.12 Logging and Monitoring Review	21
4.13 Ethical Hacking Perspective	21
4.14 Importance of Risk-Based Assessment	21
5: Prevention and Mitigation of Web Application Vulnerabilities	23
5.1 Introduction to Prevention and Mitigation	23
5.2 Secure Coding Practices	23
5.3 Input Validation and Output Encoding	23
5.4 Authentication and Authorization Best Practices	24
5.5 Secure Session Management	24
5.6 Secure File Handling	24
5.7 Protection of Sensitive Data	25
5.8 Secure Use of Third-Party Components	25
5.9 Security Configuration Management	25
5.10 Web Application Firewalls (WAFs)	26
5.11 Secure Development Lifecycle (SDLC)	26
5.12 Regular Security Testing	26
5.13 Developer Awareness and Training	26
5.14 Defense-in-Depth Approach	27
5.15 Ethical Hacking Perspective	27
6: Conclusion and References	28
6.1 Overall Conclusion	28
6.2 Key Takeaways	28
6.3 Importance in Modern Cybersecurity	28
6.4 Role of Ethical Hackers	29
6.5 Future Scope	29
6.6 Final Statement	29
POC's	31

TABLE OF CONTENTS

1. Web Application Reconnaissance.....	31
2. Web Spidering Using OWASP ZAP	33
3. Web Application Vulnerability Scanning Using SmartScanner.....	35
3. Web Application Attacks.....	38
Sniper attack	39
Pitchfork attack.....	44
Cluster bomb attack.....	46

1: INTRODUCTION TO WEB APPLICATION SECURITY

1: Introduction to Web Application Security

1.1 INTRODUCTION

Web applications have become a fundamental component of modern digital services, enabling online banking, e-commerce, healthcare systems, education platforms, and enterprise solutions. As their usage has increased, web applications have also become **primary targets for cyberattacks**.

Web application hacking refers to identifying and exploiting weaknesses in web-based systems to gain unauthorized access, manipulate data, or disrupt services. From an ethical hacking perspective, understanding these vulnerabilities is essential for **securing applications and protecting user data**.

1.2 WHAT IS A WEB APPLICATION?

A **web application** is a software program that:

- Runs on a web server
- Is accessed through a web browser
- Uses HTTP/HTTPS for communication
- Interacts with backend databases and services

Examples include:

- Online shopping portals
 - Social media platforms
 - Banking applications
 - Content management systems
-

1.3 WEB APPLICATION ARCHITECTURE OVERVIEW

A typical web application consists of:

1.3.1 Client Side

- Web browser

1: INTRODUCTION TO WEB APPLICATION SECURITY

- HTML, CSS, JavaScript
- Handles user interaction

1.3.2 Server Side

- Application logic
- Authentication and authorization
- Business rules

1.3.3 Database Layer

- Stores user data
- Maintains application records
- Supports application logic

Security weaknesses in any layer can lead to compromise.

1.4 WHY WEB APPLICATIONS ARE TARGETED

Web applications are frequently attacked because:

- They are internet-facing
- They process sensitive data
- They often rely on complex logic
- They are developed rapidly with security gaps

Attackers exploit logic flaws rather than breaking infrastructure.

1.5 COMMON SECURITY CHALLENGES IN WEB APPLICATIONS

- Improper input validation
- Weak authentication mechanisms
- Poor session management
- Insecure file handling

1: INTRODUCTION TO WEB APPLICATION SECURITY

- Misconfigured servers
- Excessive trust in client-side controls

These challenges increase attack surface.

1.6 DIFFERENCE BETWEEN NETWORK AND WEB APPLICATION ATTACKS

Aspect	Network Attacks	Web Application Attacks
--------	-----------------	-------------------------

Target	Infrastructure	Application logic
Focus	Ports & services	Input & workflows
Complexity	Lower	Higher
Detection	Easier	Harder

Web application attacks often bypass traditional network defenses.

1.7 OWASP AND WEB APPLICATION SECURITY

The **Open Web Application Security Project (OWASP)** plays a key role by:

- Identifying common web vulnerabilities
- Publishing security guidelines
- Raising developer awareness

OWASP Top 10 vulnerabilities are closely aligned with CEH Module 14 topics.

1.8 IMPACT OF WEB APPLICATION VULNERABILITIES

Successful attacks can lead to:

- Data breaches
- Account takeover
- Financial fraud

1: INTRODUCTION TO WEB APPLICATION SECURITY

- Service disruption
 - Legal and compliance issues
 - Loss of user trust
-

1.9 ETHICAL HACKING PERSPECTIVE

From an ethical standpoint:

- Testing must be authorized
- Focus is on identification and mitigation
- Goal is risk reduction, not exploitation

Ethical hackers help organizations improve resilience.

1.10 LEGAL AND COMPLIANCE CONSIDERATIONS

Web applications often handle regulated data, requiring compliance with:

- GDPR
- PCI-DSS
- HIPAA
- ISO/IEC standards

Security failures may lead to penalties.

1.11 SCOPE OF MODULE 14

This module covers:

- Web application architecture
- Common vulnerabilities
- Attack surfaces (theoretical)
- Detection techniques

1: INTRODUCTION TO WEB APPLICATION SECURITY

- Prevention and mitigation
 - Best practices
-

1.12 LEARNING OBJECTIVES

After completing this module, learners should be able to:

- Understand web application attack vectors
 - Identify common security weaknesses
 - Explain impacts of vulnerabilities
 - Recommend secure development practices
 - Align defenses with standards
-

1.13 SUMMARY OF PART 1

Part 1 introduced:

- Web application fundamentals
- Why web apps are attacked
- Security challenges and impact
- Ethical hacking role

This sets the foundation for studying vulnerabilities in detail.

2: WEB APPLICATION ARCHITECTURE AND ATTACK SURFACE

2: Web Application Architecture and Attack Surface

2.1 INTRODUCTION TO WEB APPLICATION ARCHITECTURE

Understanding web application architecture is essential for identifying where security weaknesses may exist. Web applications are composed of multiple interconnected components, and a vulnerability in any one component can compromise the entire application.

From an ethical hacking perspective, analyzing architecture helps in:

- Identifying attack entry points
 - Understanding data flow
 - Recognizing trust boundaries
-

2.2 THREE-TIER WEB APPLICATION ARCHITECTURE

Most modern web applications follow a **three-tier architecture**:

2.2.1 Presentation Layer (Client Layer)

This layer includes:

- Web browsers
- HTML, CSS, JavaScript
- User input forms

Security concerns:

- Client-side validation bypass
- Insecure JavaScript logic
- Manipulation of requests

Client-side controls should never be trusted alone.

2.2.2 Application Layer (Server Layer)

This layer handles:

2: WEB APPLICATION ARCHITECTURE AND ATTACK SURFACE

- Business logic
- Authentication and authorization
- Session handling
- Input processing

Security concerns:

- Logic flaws
- Improper input validation
- Broken authentication
- Insecure session management

Most critical vulnerabilities exist at this layer.

2.2.3 Data Layer (Database Layer)

This layer manages:

- User data
- Application records
- Transactions

Security concerns:

- SQL Injection
- Insecure database permissions
- Data exposure

Secure interaction with databases is crucial.

2.3 SUPPORTING COMPONENTS IN WEB APPLICATIONS

Beyond the core layers, web applications rely on additional components:

- Web servers (Apache, Nginx, IIS)
- Application servers

2: WEB APPLICATION ARCHITECTURE AND ATTACK SURFACE

- APIs
- Third-party services
- Cloud infrastructure

Each component expands the attack surface.

2.4 DATA FLOW IN WEB APPLICATIONS

Typical data flow:

1. User sends request
2. Server processes input
3. Business logic executes
4. Database interaction occurs
5. Response is sent to user

Security failures can occur at any stage of this flow.

2.5 TRUST BOUNDARIES IN WEB APPLICATIONS

A **trust boundary** is where data moves from a less trusted zone to a more trusted zone.

Common trust boundaries include:

- Browser to server
- Application to database
- Application to third-party APIs

Attackers target these boundaries to inject malicious input.

2.6 WHAT IS AN ATTACK SURFACE?

The **attack surface** refers to all points where an attacker can interact with an application.

In web applications, this includes:

2: WEB APPLICATION ARCHITECTURE AND ATTACK SURFACE

- Input fields
- URL parameters
- Cookies
- HTTP headers
- APIs
- File upload features

Reducing attack surface reduces risk.

2.7 CLIENT-SIDE ATTACK SURFACE

Client-side attack surface includes:

- Forms
- JavaScript functions
- Hidden fields
- Cookies and local storage

Attackers can easily manipulate client-side data.

2.8 SERVER-SIDE ATTACK SURFACE

Server-side attack surface includes:

- Authentication endpoints
- Session management mechanisms
- Business logic workflows
- File handling functions

These areas are prime targets for exploitation.

2: WEB APPLICATION ARCHITECTURE AND ATTACK SURFACE

2.9 API ATTACK SURFACE

Modern web applications rely heavily on APIs.

API-related risks include:

- Broken authentication
- Excessive data exposure
- Improper authorization
- Lack of rate limiting

APIs must be secured independently.

2.10 THIRD-PARTY AND DEPENDENCY RISKS

Web applications often integrate:

- Payment gateways
- Analytics services
- Authentication providers

Security issues in third-party components can directly impact the application.

2.11 CONFIGURATION-BASED ATTACK SURFACE

Misconfigurations create vulnerabilities such as:

- Default credentials
- Debug mode enabled
- Directory listing
- Insecure headers

Configuration errors are common and dangerous.

2: WEB APPLICATION ARCHITECTURE AND ATTACK SURFACE

2.12 IMPORTANCE OF ARCHITECTURAL UNDERSTANDING FOR ETHICAL HACKERS

Ethical hackers use architectural knowledge to:

- Identify high-risk components
 - Prioritize testing efforts
 - Understand attack paths
 - Recommend architectural improvements
-

2.13 Summary of Part 2

Part 2 covered:

- Web application architecture
- Core layers and components
- Trust boundaries
- Attack surface identification

This architectural understanding is essential before studying specific web application vulnerabilities.

3: COMMON WEB APPLICATION VULNERABILITIES

3: Common Web Application Vulnerabilities

3.1 INTRODUCTION TO WEB APPLICATION VULNERABILITIES

Web application vulnerabilities arise due to:

- Insecure coding practices
- Complex application logic
- Rapid development cycles
- Inadequate security testing

Most web application attacks exploit **logic and input-handling flaws**, not infrastructure weaknesses.

3.2 OWASP TOP 10 AND CEH ALIGNMENT

The **OWASP Top 10** identifies the most critical web application security risks. CEH Module 14 closely aligns with these categories to help ethical hackers understand real-world vulnerabilities.

3.3 INJECTION VULNERABILITIES

Injection vulnerabilities occur when untrusted input is interpreted as commands.

Types of Injection

- SQL Injection
- Command Injection
- LDAP Injection
- XPath Injection

Root Cause:

- Lack of input validation
- Dynamic query construction

Impact:

- Data compromise

3: COMMON WEB APPLICATION VULNERABILITIES

- Authentication bypass
 - System manipulation
-

3.4 BROKEN AUTHENTICATION

Broken authentication occurs when applications:

- Use weak credential management
- Implement insecure login mechanisms
- Fail to protect authentication tokens

Common issues include:

- Weak passwords
- Credential reuse
- Improper session handling

Impact:

- Account takeover
 - Unauthorized access
-

3.5 BROKEN ACCESS CONTROL

Broken access control occurs when users can:

- Access unauthorized resources
- Perform actions beyond their privileges

Examples (Conceptual):

- Viewing another user's data
- Modifying restricted resources

Impact:

- Data leakage
-

3: COMMON WEB APPLICATION VULNERABILITIES

- Privilege escalation
-

3.6 SECURITY MISCONFIGURATION

Security misconfiguration results from:

- Default settings
- Unnecessary services enabled
- Debug features left active

Examples include:

- Directory listing enabled
 - Verbose error messages
 - Unsecured admin panels
-

3.7 SENSITIVE DATA EXPOSURE

Sensitive data exposure occurs when:

- Data is not encrypted properly
- Weak cryptographic mechanisms are used

Affected data includes:

- Credentials
- Personal information
- Financial data

Impact:

- Identity theft
- Regulatory violations

3: COMMON WEB APPLICATION VULNERABILITIES

3.8 CROSS-SITE SCRIPTING (XSS)

XSS occurs when applications include untrusted input in web pages.

Types of XSS

- Stored XSS
- Reflected XSS
- DOM-based XSS

Impact:

- Session hijacking
 - Credential theft
 - Defacement
-

3.9 INSECURE DESERIALIZATION

Insecure deserialization occurs when:

- Applications trust serialized data
- Malicious objects are processed

Impact:

- Remote code execution
 - Privilege escalation
 - Application compromise
-

3.10 USING COMPONENTS WITH KNOWN VULNERABILITIES

Applications often rely on:

- Libraries
- Frameworks
- Plugins

3: COMMON WEB APPLICATION VULNERABILITIES

Outdated components may contain known vulnerabilities.

Impact:

- Indirect compromise
 - Supply chain risk
-

3.11 INSUFFICIENT LOGGING AND MONITORING

Lack of logging allows:

- Attacks to go undetected
- Delayed incident response

Impact:

- Increased attacker dwell time
 - Greater damage
-

3.12 BUSINESS LOGIC FLAWS

Business logic flaws exploit:

- Incorrect assumptions
- Improper workflow enforcement

These flaws are:

- Application-specific
 - Hard to detect automatically
-

3.13 CLIENT-SIDE SECURITY WEAKNESSES

Client-side weaknesses include:

- Trusting client input
 - Insecure JavaScript logic
-

3: COMMON WEB APPLICATION VULNERABILITIES

Client-side validation can be bypassed easily.

3.14 IMPACT OF COMBINED VULNERABILITIES

Multiple vulnerabilities combined can:

- Amplify attack impact
- Enable multi-stage attacks

Defense must address vulnerabilities holistically.

3.15 ETHICAL HACKING PERSPECTIVE

Ethical hackers:

- Identify vulnerability patterns
 - Assess real-world impact
 - Recommend prioritized remediation
-

3.16 Summary of Part 3

Part 3 covered:

- Major web application vulnerabilities
- OWASP-aligned risk categories
- Root causes and impacts

Understanding these vulnerabilities is critical before studying detection and prevention.

4: DETECTION AND ASSESSMENT OF WEB APPLICATION VULNERABILITIES

4: Detection and Assessment of Web Application Vulnerabilities

4.1 INTRODUCTION TO WEB APPLICATION VULNERABILITY DETECTION

Detecting web application vulnerabilities is a critical step in securing applications. Unlike network vulnerabilities, web application flaws are often embedded within **application logic, workflows, and data handling mechanisms**, making detection more complex.

Effective detection helps organizations:

- Prevent data breaches
 - Reduce attack surface
 - Improve application resilience
-

4.2 CHALLENGES IN WEB APPLICATION VULNERABILITY DETECTION

Detection is challenging due to:

- Encrypted web traffic (HTTPS)
- Complex application logic
- Frequent application updates
- Dynamic content
- Use of third-party components

Many vulnerabilities do not generate obvious error messages.

4.3 WEB APPLICATION RECONNAISSANCE

Reconnaissance involves understanding:

- Application structure
 - Available functionality
-

4: DETECTION AND ASSESSMENT OF WEB APPLICATION VULNERABILITIES

- User roles and permissions
- Input points and workflows

This phase focuses on **mapping the application**, not exploitation.

4.4 INPUT VALIDATION AND DATA FLOW ANALYSIS

Analyzing how data flows through the application helps identify:

- Missing input validation
- Unsafe data handling
- Trust boundary violations

Understanding data flow is crucial for detecting injection and logic flaws.

4.5 AUTHENTICATION AND AUTHORIZATION TESTING

Assessment focuses on:

- Login mechanisms
- Password policies
- Session handling
- Access controls

Weaknesses here often lead to account compromise.

4.6 SESSION MANAGEMENT ANALYSIS

Session-related assessment examines:

- Session token handling
- Session expiration
- Secure cookie attributes

Poor session management can enable hijacking and fixation attacks.

4: DETECTION AND ASSESSMENT OF WEB APPLICATION VULNERABILITIES

4.7 ERROR HANDLING AND EXCEPTION ANALYSIS

Improper error handling may:

- Reveal internal details
- Leak database or server information

Secure applications handle errors gracefully.

4.8 BUSINESS LOGIC ASSESSMENT

Business logic testing involves:

- Reviewing workflows
- Identifying missing controls
- Checking transaction integrity

Logic flaws are often unique to the application.

4.9 CLIENT-SIDE CODE REVIEW

Client-side assessment includes:

- JavaScript analysis
- Validation logic review
- Exposure of sensitive information

Client-side code should not enforce security decisions alone.

4.10 API SECURITY ASSESSMENT

API assessment focuses on:

- Authentication mechanisms
- Authorization checks
- Data exposure

4: DETECTION AND ASSESSMENT OF WEB APPLICATION VULNERABILITIES

- Rate limiting

APIs are a growing attack vector.

4.11 CONFIGURATION AND DEPLOYMENT REVIEW

Assessment includes checking for:

- Default configurations
- Unnecessary features enabled
- Insecure headers

Misconfigurations are common and exploitable.

4.12 LOGGING AND MONITORING REVIEW

Assessing logging mechanisms helps determine:

- Whether attacks are detectable
- Incident response readiness

Insufficient logging increases risk.

4.13 ETHICAL HACKING PERSPECTIVE

Ethical hackers:

- Perform structured assessments
 - Document findings clearly
 - Prioritize vulnerabilities by risk
 - Provide remediation guidance
-

4.14 IMPORTANCE OF RISK-BASED ASSESSMENT

Not all vulnerabilities have equal impact. Risk assessment considers:

4: DETECTION AND ASSESSMENT OF WEB APPLICATION VULNERABILITIES

- Likelihood of exploitation
- Potential impact
- Asset criticality

Risk-based prioritization improves security outcomes.

4.15 Summary of Part 4

Part 4 covered:

- Web application vulnerability detection concepts
- Assessment techniques
- Importance of logic and workflow analysis

This prepares us for prevention and mitigation strategies.

5: PREVENTION AND MITIGATION OF WEB APPLICATION VULNERABILITIES

5: Prevention and Mitigation of Web Application Vulnerabilities

5.1 INTRODUCTION TO PREVENTION AND MITIGATION

Preventing web application vulnerabilities requires a **secure-by-design approach**, where security is integrated throughout the **application development and deployment lifecycle**. Mitigation strategies aim to reduce risk even when vulnerabilities exist.

Effective prevention minimizes:

- Data breaches
 - Account compromise
 - Service disruption
 - Legal and compliance risks
-

5.2 SECURE CODING PRACTICES

Secure coding is the foundation of web application security.

Key principles include:

- Treat all user input as untrusted
- Validate input on the server side
- Use safe APIs and frameworks
- Avoid custom security implementations

Secure coding prevents most common vulnerabilities.

5.3 INPUT VALIDATION AND OUTPUT ENCODING

Applications should:

- Validate input type, length, and format
 - Reject unexpected input
-

5: PREVENTION AND MITIGATION OF WEB APPLICATION VULNERABILITIES

- Encode output based on context

Proper validation and encoding prevent injection and XSS attacks.

5.4 AUTHENTICATION AND AUTHORIZATION BEST PRACTICES

Strong authentication includes:

- Secure password policies
- Multi-factor authentication
- Protection against brute-force attacks

Authorization best practices include:

- Role-based access control
 - Server-side enforcement
 - Regular access reviews
-

5.5 SECURE SESSION MANAGEMENT

Secure session management requires:

- Strong, random session identifiers
- Secure cookie attributes (HttpOnly, Secure)
- Proper session expiration
- Session invalidation on logout

This reduces session-related attacks.

5.6 SECURE FILE HANDLING

File upload and download features should:

- Validate file types
 - Enforce size limits
-

5: PREVENTION AND MITIGATION OF WEB APPLICATION VULNERABILITIES

- Store files securely
- Prevent execution of uploaded content

Improper file handling can lead to severe compromise.

5.7 PROTECTION OF SENSITIVE DATA

Sensitive data should be:

- Encrypted in transit and at rest
- Stored using strong cryptographic algorithms
- Accessed only by authorized components

Data protection is critical for compliance.

5.8 SECURE USE OF THIRD-PARTY COMPONENTS

Organizations should:

- Maintain an inventory of dependencies
- Monitor for known vulnerabilities
- Apply updates regularly

Supply chain security is essential.

5.9 SECURITY CONFIGURATION MANAGEMENT

Secure configuration includes:

- Disabling unnecessary features
- Enforcing secure defaults
- Applying security headers

Configuration errors are a major attack vector.

5: PREVENTION AND MITIGATION OF WEB APPLICATION VULNERABILITIES

5.10 WEB APPLICATION FIREWALLS (WAFS)

WAFs provide:

- An additional layer of protection
- Detection of known attack patterns

They should complement secure coding, not replace it.

5.11 SECURE DEVELOPMENT LIFECYCLE (SDLC)

Security should be integrated into:

- Requirements
- Design
- Development
- Testing
- Deployment
- Maintenance

A secure SDLC reduces long-term risk.

5.12 REGULAR SECURITY TESTING

Testing methods include:

- Code reviews
- Vulnerability assessments
- Penetration testing

Regular testing ensures continued security.

5.13 DEVELOPER AWARENESS AND TRAINING

Developers must:

5: PREVENTION AND MITIGATION OF WEB APPLICATION VULNERABILITIES

- Understand common vulnerabilities
- Follow secure coding standards
- Stay updated on security practices

Human factors are critical.

5.14 DEFENSE-IN-DEPTH APPROACH

Web application security should include:

- Application security controls
- Network security controls
- Monitoring and logging
- Incident response capabilities

Layered defense improves resilience.

5.15 ETHICAL HACKING PERSPECTIVE

Ethical hackers:

- Validate mitigation effectiveness
 - Identify residual risks
 - Provide actionable recommendations
-

5.16 Summary of Part 5

Part 5 covered:

- Secure coding principles
- Preventive controls
- Mitigation strategies
- Defense-in-depth approach

6: CONCLUSION AND REFERENCES

6: Conclusion and References

6.1 OVERALL CONCLUSION

This report provided a comprehensive theoretical analysis of **web application security and hacking concepts** as covered in **CEH v13 Module 14**. It examined web application architecture, attack surfaces, common vulnerabilities, detection and assessment methods, and prevention and mitigation strategies.

The study highlights that web application attacks primarily exploit **logic flaws, improper input handling, weak authentication, and misconfigurations**, rather than weaknesses in underlying network infrastructure. As web applications continue to evolve, security must be treated as a **core design requirement**, not an afterthought.

6.2 KEY TAKEAWAYS

The main conclusions drawn from this module include:

- Web applications are high-value targets due to internet exposure.
 - Most vulnerabilities originate from insecure coding practices.
 - Client-side controls alone cannot enforce security.
 - Detection requires deep understanding of application logic.
 - Prevention is most effective when integrated into the SDLC.
 - Defense-in-depth significantly improves application resilience.
-

6.3 IMPORTANCE IN MODERN CYBERSECURITY

Web application security is critical because:

- Applications handle sensitive user and business data
- Attacks can bypass traditional network defenses
- Vulnerabilities can lead to large-scale breaches
- Regulatory and compliance requirements demand strong protection

Modern organizations must prioritize application-layer security.

6: CONCLUSION AND REFERENCES

6.4 ROLE OF ETHICAL HACKERS

Ethical hackers contribute by:

- Identifying web application vulnerabilities
- Assessing real-world risk
- Validating security controls
- Supporting secure development practices

Their role ensures vulnerabilities are discovered before attackers exploit them.

6.5 FUTURE SCOPE

Future web application security efforts will focus on:

- Secure-by-default frameworks
- Automated vulnerability detection
- AI-assisted code analysis
- Stronger API security models
- Zero Trust architectures

Continuous improvement remains essential.

6.6 FINAL STATEMENT

Web application hacking remains a persistent challenge due to the complexity of modern applications. However, by applying secure coding practices, regular testing, strong authentication, and layered defenses, organizations can significantly reduce their risk exposure.

This module reinforces the principle that **web application security is an ongoing process**, requiring technical controls, developer awareness, and organizational commitment.

6.7 References

Books

1. Stallings, W. – *Network Security Essentials*

6: CONCLUSION AND REFERENCES

2. Easttom, C. – *Computer Security Fundamentals*

Certification Material

3. EC-Council – *Certified Ethical Hacker (CEH) v13 Official Courseware, Module 14*

Standards and Guidelines

4. OWASP – *OWASP Top 10 Web Application Security Risks*
5. NIST SP 800-53 – *Security and Privacy Controls*
6. ISO/IEC 27001 – *Information Security Management Systems*

Technical Resources

7. SANS Institute – *Web Application Security Whitepapers*

W3C – *Web Security Standards Documentation*

POC's

1. WEB APPLICATION RECONNAISSANCE

`nmap -T4 -A -v www.moviescope.com`

Explanation:

- `-T4` → Faster scan timing (aggressive but stable)
- `-A` → Enables OS detection, version detection, script scanning, and traceroute
- `-v` → Verbose output (shows detailed scan progress)

`www.moviescope.com` → Target web application

POC'S

```
kali linux [Running] - Oracle VirtualBox
File Machine View Input Devices Help

root@aviku: /home/aviku

Session Actions Edit View Help
zsh: corrupt history file /home/aviku/.zsh_history
[aviku@aviku]~$ sudo su
[sudo] password for aviku:
[aviku@aviku]~$ nmap -T4 -A -v www.moviescope.com

Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-25 01:28 EST
Nmap wishes you a merry Christmas! Specify -sx for Xmas Scan (https://nmap.org/book/man-port-scanning-techniques.html).
NSE: Loaded 157 scripts for scanning.
NSE: Script Pre-scanning.
Initiating NSE at 01:28
Completed NSE at 01:28, 0.00s elapsed
Initiating NSE at 01:28
Completed NSE at 01:28, 0.00s elapsed
Initiating NSE at 01:28
Completed NSE at 01:28, 0.00s elapsed
Initiating Ping Scan at 01:28
Scanning www.moviescope.com (172.233.219.78) [4 ports]
Completed Ping Scan at 01:28, 0.03s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 01:28
Completed Parallel DNS resolution of 1 host. at 01:28, 0.36s elapsed
Initiating SYN Stealth Scan at 01:28
Scanning www.moviescope.com (172.233.219.78) [1000 ports]
Discovered open port 443/tcp on 172.233.219.78
Discovered open port 80/tcp on 172.233.219.78
Discovered open port 80/tcp on 172.233.219.78
Discovered open port 443/tcp on 172.233.219.78
Completed SYN Stealth Scan at 01:29, 18.89s elapsed (1000 total ports)
Initiating Service scan at 01:29
Scanning 2 services on www.moviescope.com (172.233.219.78)
Completed Service scan at 01:29, 29.39s elapsed (2 services on 1 host)
Initiating OS detection (try #1) against www.moviescope.com (172.233.219.78)
Retrying OS detection (try #2) against www.moviescope.com (172.233.219.78)
Initiating Traceroute at 01:29
Completed Traceroute at 01:29, 0.02s elapsed
Initiating Parallel DNS resolution of 1 host. at 01:29
Completed Parallel DNS resolution of 1 host. at 01:29, 0.00s elapsed
NSE: Script scanning 172.233.219.78.
Initiating NSE at 01:29
Completed NSE at 01:30, 23.97s elapsed
```

```
kali linux [Running] - Oracle VirtualBox
File Machine View Input Devices Help

root@aviku: /home/aviku

Session Actions Edit View Help
|_http-title: 400 The plain HTTP request was sent to HTTPS port
|_ssl-cert: Subject: commonName=*.moviescope.com
| Subject Alternative Names: DNS:*.moviescope.com
| Issuer: commonName=R13/organizationName=Let's Encrypt/countryName=US
| Public Key type: rsa
| Public Key bits: 2048
| Signature Algorithm: sha256withRSAEncryption
| Not valid before: 2025-10-31T17:29:20
| Not valid after: 2026-01-20T17:29:19
| MD5: 6ad1:6eb1:f96b:bea4:5858:7793:b9d4:ad6b
| SHA-1: e8a9:0a76:5582:42d5:0b66:85db:5efc:3a3a:078c:6aad
Warning: OSscan results may be unreliable because we could not find at least 1 open and 1 closed port
Device type: general purpose
Running (JUST GUESSING): Linux 4.X|5.X (87%)
OS CPE: cpe:/o:linux:linux_kernel:cpe:/o:linux:linux_kernel:5
Aggressive OS guesses: Linux 4.15 - 5.19 (87%)
No exact OS matches for host (test conditions non-ideal).
Uptime guess: 0.000 days (since Thu Dec 25 01:29:43 2025)
Network Distance: 2 hops
TCP Sequence Prediction: Difficulty=260 (Good luck!)
IP ID Sequence Generation: All zeros

TRACEROUTE (using port 80/tcp)
HOP RTT ADDRESS
1 0.70 ms Unit (192.168.1.1)
2 4.26 ms k8s-svc-lander-shared-02.us-ord.parklogic.net (172.233.219.78)

NSE: Script Post-scanning.
Initiating NSE at 01:30
Completed NSE at 01:30, 0.00s elapsed
Initiating NSE at 01:30
Completed NSE at 01:30, 0.00s elapsed
Initiating NSE at 01:30
Completed NSE at 01:30, 0.00s elapsed
Read data files from: /usr/share/nmap
OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 85.44 seconds
Raw packets sent: 3119 (142.320KB) | Rcvd: 65 (3.212KB)

[aviku@aviku]~$
```

telnet www.moviescope.com 80

POC'S

```
kali linux [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help

root@aviku: /home/aviku
telnet certifiedhacker.com 80

Trying 162.241.216.11...
Connected to certifiedhacker.com.
Escape character is '^]'.
GET / HTTP/1.0
HTTP/1.1 408 Request Timeout
Date: Thu, 25 Dec 2025 06:37:09 GMT
Server: Apache
Content-Length: 346
Connection: close
Content-Type: text/html; charset=iso-8859-1

<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>408 Request Timeout</title>
</head><body>
<h1>Request Timeout</h1>
<p>Server timeout waiting for the HTTP request from the client.</p>
<p>Additionally, a 408 Request Timeout
error was encountered while trying to use an ErrorDocument to handle the request.</p>
</body></html>
Connection closed by foreign host.

root@aviku: /home/aviku
```

2. WEB SPIDERING USING OWASP ZAP

```
kali linux [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help

root@aviku: /home/aviku
zapproxy
Found Java version 21.0.8
Available memory: 4952 MB
Using JVM args: -Xmx1238m
1493 [main] INFO org.parosproxy.paros.Con
1899 [main] INFO org.parosproxy.paros.Con
1900 [main] INFO org.parosproxy.paros.Con
1901 [main] INFO org.parosproxy.paros.Con
1901 [main] INFO org.parosproxy.paros.Con
1990 [main] WARN org.zaproxy.zap.ZapBoots
2023 [main] INFO org.zaproxy.zap.GuiBoots
2365 [AWT-EventQueue-0] WARN org.zaproxy..
ng sun.awt.X11.XToolkit.awtAppClassName ac
4903 [AWT-EventQueue-0] INFO org.parospro
11291 [ZAP-BootstrapGUI] INFO org.zaproxy
version=71.0.0], [id=authhelper, version=0
, [id=client, version=0.15.0], [id=commonl
ion=9.0.0], [id=domxss, version=21.0.0], [
on=13.15.0], [id=gettingStarted, version=1
version=0.19.0], [id=invoke, version=16.0.
pi, version=45.0.0], [id=postman, version=
dreplace, version=20.0.0], [id=reports, '
], [id=reveal, version=9.0.0], [id=scanpol
ion=8.0.0], [id=soap, version=24.0.0], [id=spider, version=0.14.0], [id=spiderAjax, version=23.23.0], [id=tips, version=14.0.0], [id=webdriverlin
ux, version=131.0.0], [id=websocket, version=32.0.0], [id=zest, version=48.5.0]]
11311 [ZAP-BootstrapGUI] INFO org.zaproxy.zap.control.ExtensionFactory - Loading extensions
13219 [ZAP-BootstrapGUI] INFO org.zaproxy.addon.network.internal.TlsUtils - Using supported SSL/TLS protocols: [TLSv1.2, TLSv1.3]
13737 [ZAP-BootstrapGUI] INFO org.zaproxy.zap.control.ExtensionFactory - Extensions loaded

Welcome to ZAP!
ZAP is an easy to use integrated penetration testing tool for
finding vulnerabilities in web applications.
If you are new to ZAP then it is best to start with one of the
options below.

Recommended!
t/.ZAP/ cores: 3 maxMemory: 1 GB
make field private static java.lang.Stri
unnamed module @2f490758

filters, version=23.0.0], [id=ascanrules,
on=17.0.0], [id=callhome, version=0.14.0]
ersion=17.0.0], [id=directorylistv1, vers
mhandler, version=6.7.0], [id=fuzz, versi
.0], [id=help, version=20.0.0], [id=hud,
id=onlineMenu, version=14.0.0], [id=opena
0.0], [id=quickstart, version=51.0.0], [i
rsion=0.11.0], [id=retire, version=0.46.0
ium, version=15.36.0], [id=sequence, vers
ion=14.0.0], [id=webdriverlin
```

POC'S

The screenshot shows the ZAP Automated Scan interface. The 'URL to attack' is set to `http://testfire.net`. The 'Use traditional spider' checkbox is checked. The 'Use ajax spider' dropdown is set to 'If Modern' with 'Firefox' selected. The 'Attack' button is highlighted. The 'Progress' bar shows 'Actively scanning (attacking) the URLs discovered by the spider(s)'. Below the scan controls, a table lists the discovered URLs and their response details.

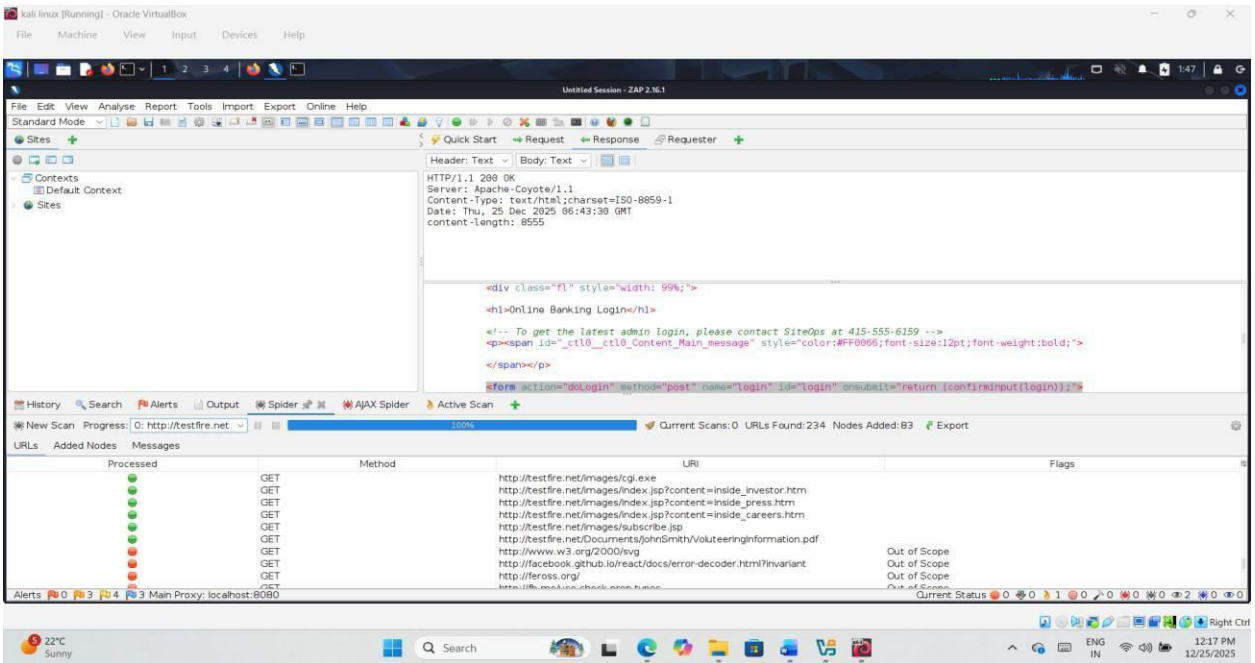
ID	Req. Timestamp	Resp. Timestamp	Method	URL	Code	Reason	RTT	Size Resp. Header	Size Resp. Body
932	12/25/25, 1:46:18 AM	12/25/25, 1:46:18 AM	GET	http://testfire.net/robots.txt	200	OK	297 ms	126 bytes	0 bytes
933	12/25/25, 1:46:18 AM	12/25/25, 1:46:18 AM	POST	http://testfire.net/doSubscribe	302	Found	305 ms	126 bytes	0 bytes
934	12/25/25, 1:46:18 AM	12/25/25, 1:46:18 AM	GET	http://testfire.net/disclaimer.htm?url=http%3A%2F%2Ftestfire.net%2FdoLogin	200	OK	303 ms	226 bytes	2,083 bytes
935	12/25/25, 1:46:18 AM	12/25/25, 1:46:18 AM	GET	http://testfire.net/PrivacyPolicy.jsp?sec=CareersSite	404	Not Found	309 ms	156 bytes	6,958 bytes
936	12/25/25, 1:46:18 AM	12/25/25, 1:46:18 AM	POST	http://testfire.net/sendFeedback	200	OK	305 ms	149 bytes	7,221 bytes
937	12/25/25, 1:46:18 AM	12/25/25, 1:46:18 AM	POST	http://testfire.net/doLogin	302	Found	315 ms	126 bytes	0 bytes
938	12/25/25, 1:46:18 AM	12/25/25, 1:46:18 AM	GET	http://testfire.net/disclaimer.htm?url=http%3A%2F%2Ftestfire.net%2Fimages	200	OK	297 ms	226 bytes	2,083 bytes
939	12/25/25, 1:46:18 AM	12/25/25, 1:46:18 AM	GET	http://testfire.net/images	302	Found	298 ms	125 bytes	0 bytes
940	12/25/25, 1:46:18 AM	12/25/25, 1:46:18 AM	GET	http://testfire.net/PrivacyPolicy.jsp?sec=CareersSite	404	Not Found	306 ms	156 bytes	6,958 bytes
941	12/25/25, 1:46:18 AM	12/25/25, 1:46:18 AM	POST	http://testfire.net/sendFeedback	200	OK	305 ms	149 bytes	7,224 bytes

The screenshot shows the ZAP Alerts list. The 'Alerts (10)' tab is selected. The list of alerts includes:

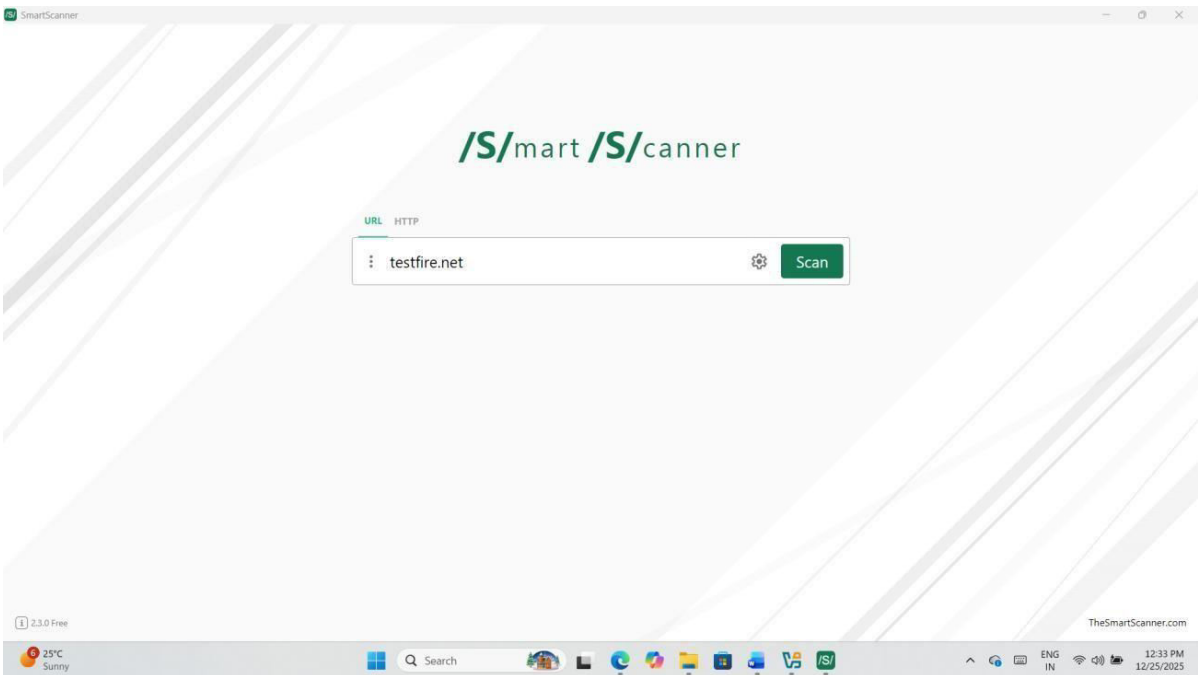
- Absence of Anti-CSRF Tokens (3)
- GET: http://testfire.net/feedback.jsp
- GET: http://testfire.net/login.jsp
- GET: http://testfire.net/subscribe.jsp
- Content Security Policy (CSP) Header Not Set
- Missing Anti-clickjacking Header (62)
- Cookie without SameSite Attribute (3)
- Cross-Domain JavaScript Source File Inclusion
- Server Leaks Version Information via "Server"
- X-Content-Type-Options Header Missing (3)
- Information Disclosure - Suspicious

The 'Main Proxy: localhost:8080' is listed at the bottom of the alerts list.

POC'S



3. WEB APPLICATION VULNERABILITY SCANNING USING SMARTSCANNER



POC'S

Common Weakness Enumeration

CWE-319: Cleartext Transmission of Sensitive Information (4.19)

Weakness ID: 319
Vulnerability Mapping: ALLOWED
Abstraction: Base

Description

The product transmits sensitive or security-critical data in cleartext in a communication channel that can be sniffed by unauthorized actors.

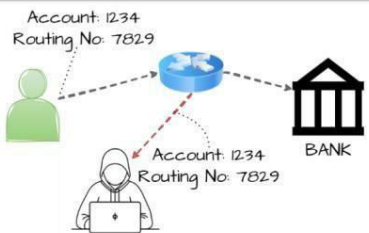
Common Consequences

Impact Details
Scope: Integrity, Confidentiality

Read Application Data; Modify Files or Directories

Anyone can read the information by gaining access to the channel being used for communication. Many communication channels can be "sniffed" (monitored) by adversaries during data transmission. For example, in networking, packets can traverse many intermediary nodes from the source to the destination, whether across the internet, an internal network, the cloud, etc. Some actors might have privileged access to a network interface or any link along the channel, such as a router, but they might not be authorized to collect the underlying data. As a result, network traffic could be sniffed by adversaries, spilling security-critical data.

Scope: Integrity, Confidentiality



SmartScanner

TARGET: testfire.net

RISK: 4.1 /5

ISSUES: 16

DURATION: 5' 14"

REQUESTS: 143

LAST REQUEST: testfire.net/undf feedback

Found Issues

Issue	Count
Cross Site Scripting	3
Session Cookie without SameSite Flag	1
Session Cookie without Secure Flag	1
No Redirection from HTTP to HTTPS	1
No HTTPS	1
Content-Security-Policy Header is Missing	1
X-Frame-Options Header is Missing	1
http://testfire.net	
Server Version Disclosure	1
Internal Server Error	1
X-Content-Type-Options Header is Missing	1
Missing or Insecure Cache-Control Header	1

Issue added: Cross Site Scripting

X-Frame-Options Header is Missing

Low

DESCRIPTION

The absence of the **X-Frame-Options** HTTP response header leaves a website vulnerable to click-jacking attacks. Without this header, attackers can embed the site's content into malicious pages using iframes, potentially leading to phishing attacks or unauthorized transactions.

RECOMMENDATION

To mitigate this vulnerability, configure your server to send the **X-Frame-Options** header with an appropriate setting for all pages. Common settings include **DENY**, **SAMEORIGIN**, or **ALLOW-FROM** followed by a specific URL. Choose the setting that best fits your application's requirements. Ensure proper testing to verify that the header is correctly implemented and enforced by all browsers.

CLASSIFICATIONS

CWE-1021 CWE-16 OWASP 2010-A6 OWASP 2013-A5 OWASP 2017-A6 OWASP 2021-A4 OWASP 2021-A5

REFERENCES

- Mozilla: Web Security
- OWASP: Clickjacking

POC'S

SmartScanner

TARGET: testfire.net

RISK: 4.3 /5

ISSUES: 23

DURATION: 5' 54"

REQUESTS: 316

LAST REQUEST: testfire.net/defuse/bee

Found Issues	
① No HTTPS	1
① Content-Security-Policy Header is Missing	1
① X-Frame-Options Header is Missing	1
① Server Version Disclosure	1
① Tomcat Version Disclosure	1
② Broken Link	2
② Content Character Encoding is not Defined	1
② X-Content-Type-Options Header is Missing	1
http://testfire.net	
② Missing or Insecure Cache-Control Header	1
② Referrer-Policy Header is Missing	1
② Target Information	1

Issue added: Vulnerable Tomcat Version

← X-Content-Type-Options Header is Missing

Informational

DESCRIPTION

The absence of the `X-Content-Type-Options` response HTTP header may expose a website to MIME sniffing attacks. MIME sniffing, performed by browsers when the MIME type is not explicitly declared, can lead to the interpretation of non-executable content as executable, potentially exposing users to security risks.

RECOMMENDATION

To mitigate this risk, configure your server to send the `X-Content-Type-Options` header with the value set to `nosniff`. This instructs browsers not to perform MIME sniffing and to strictly respect the declared content type.

CLASSIFICATIONS

CWE-16 OWASP 2010-A6 OWASP 2013-A5 OWASP 2017-A6 OWASP 2021-A5

REFERENCES

- Mozilla: Web Security
- Mozilla: X-Content-Type-Options
- Mozilla: MIME sniffing
- Mozilla: MIME types (IANA media types)

3. WEB APPLICATION ATTACKS

The screenshot shows a Kali Linux virtual machine running Oracle VM VirtualBox. The browser window displays the AltoroMutual login page. The page has a header with the AltoroMutual logo and navigation links. The main content area is titled 'Online Banking Login' and shows a 'Login Failed' message: 'Login Failed: We're sorry, but this username or password was not found in our system. Please try again.' The login form has fields for 'Username' (containing 'admin') and 'Password', and a 'Login' button. The footer of the page includes links for Privacy Policy, Security Statement, Server Status Check, and REST API, along with a copyright notice for 2025 Altoro Mutual, Inc.

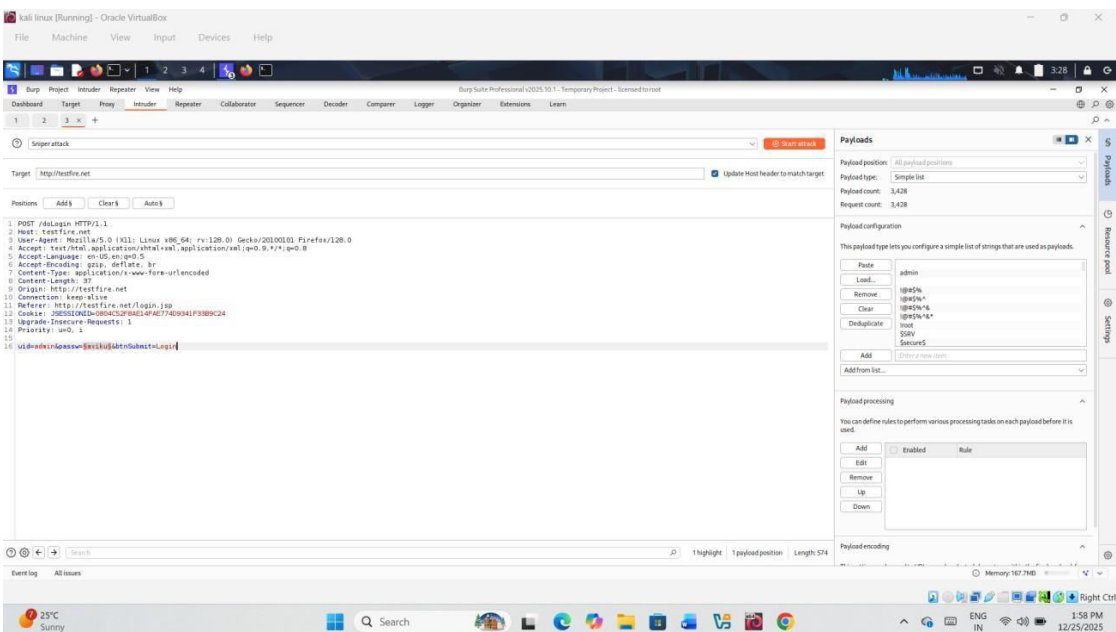
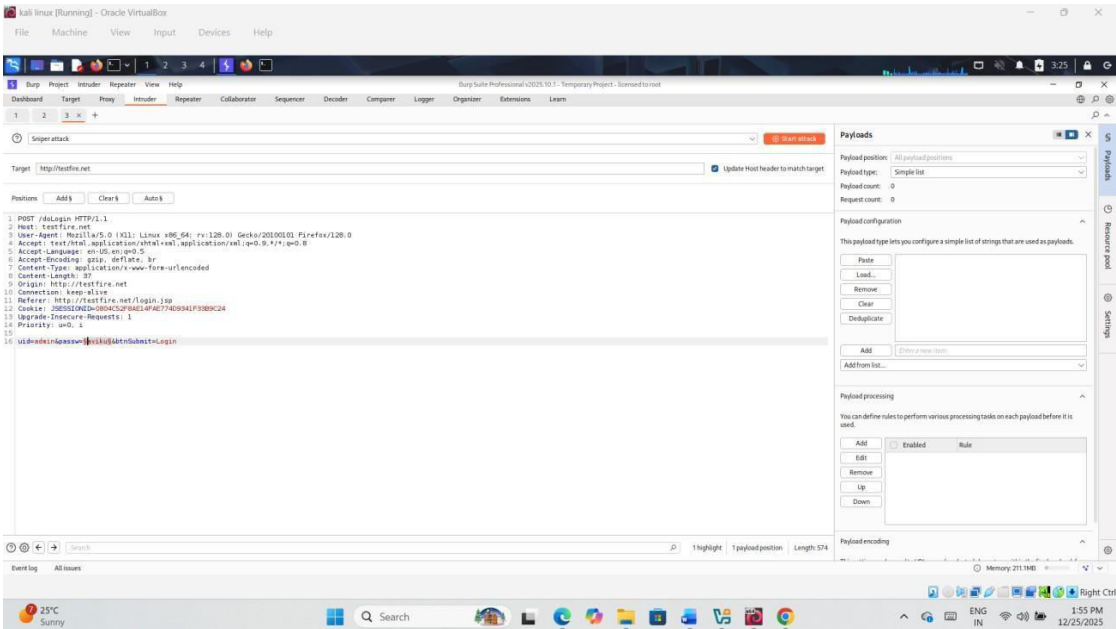
Below the browser window, the Burp Suite Professional v2025.10.1 interface is visible. The 'Intercept' tab is active, showing a list of intercepted HTTP requests. The first request is a GET request to 'http://testfire.net/login.jsp' with a status code of 200. The second request is a POST request to 'http://testfire.net/login.jsp' with a status code of 401. The 'Request' tab is selected, showing the details of the intercepted POST request. The request is a POST to 'http://testfire.net/login.jsp' with a content type of 'application/x-www-form-urlencoded'. The request body is 'username=admin&password=password'. The 'Inspector' tab is also visible, showing the request details.

At the bottom of the screenshot, the Windows taskbar is visible, showing the system clock at 1:55 PM on 12/25/2025.

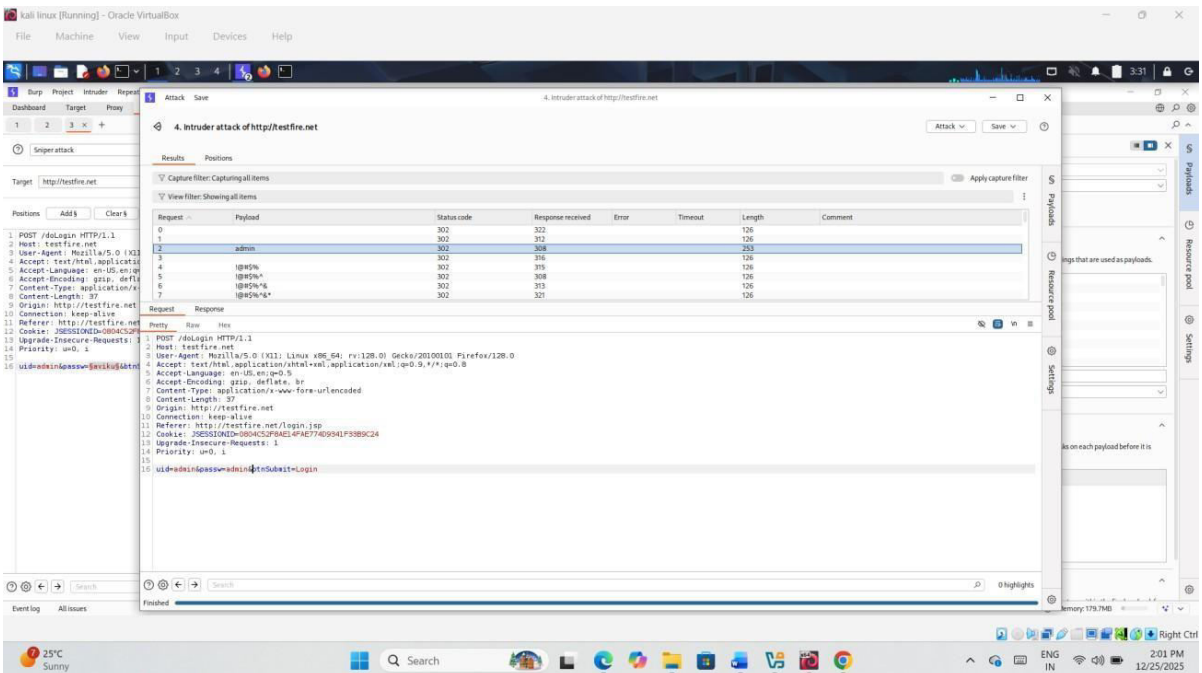
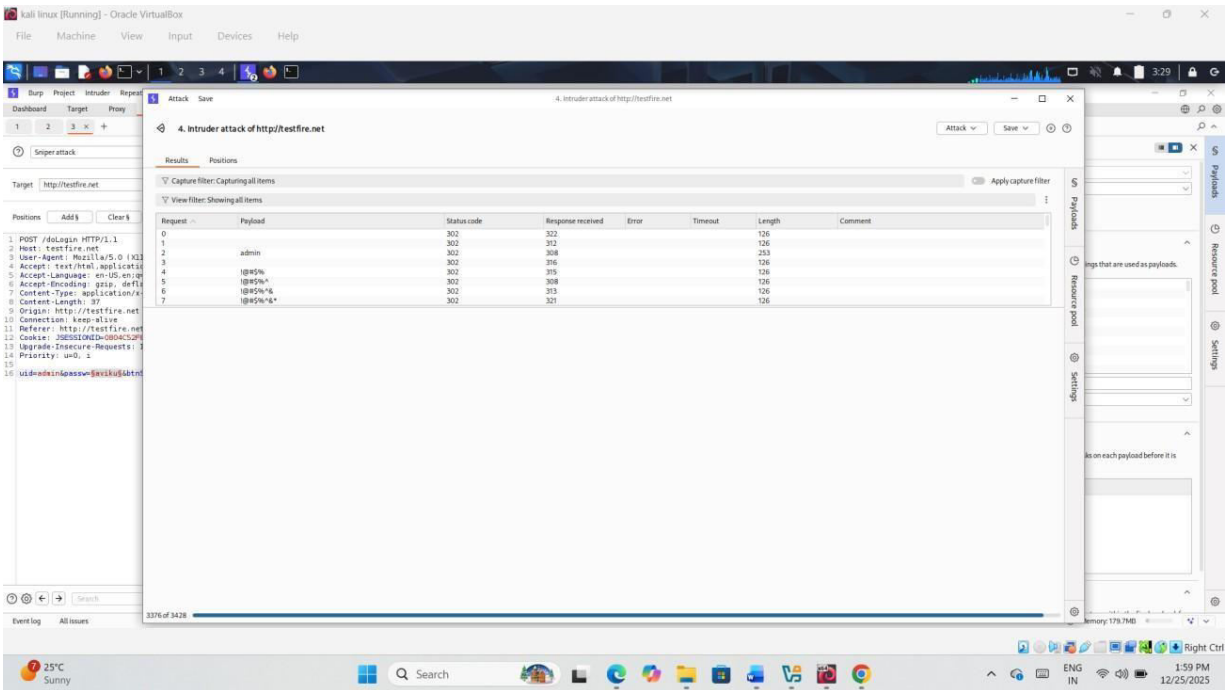
POC'S

Sniper attack

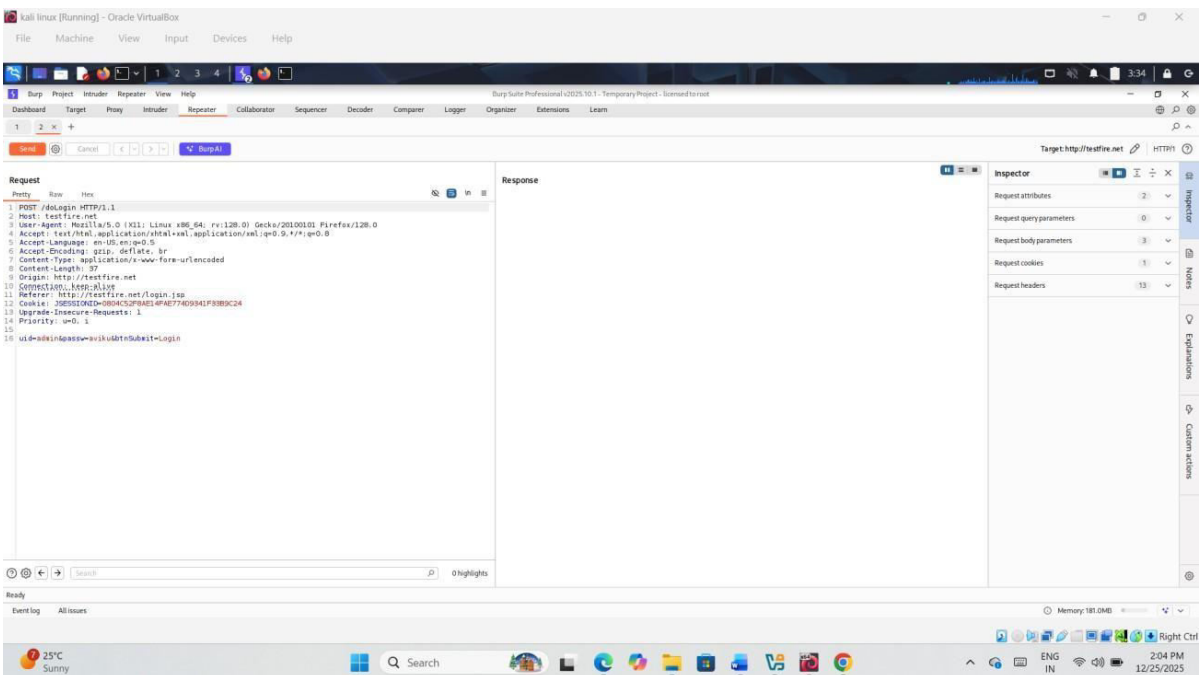
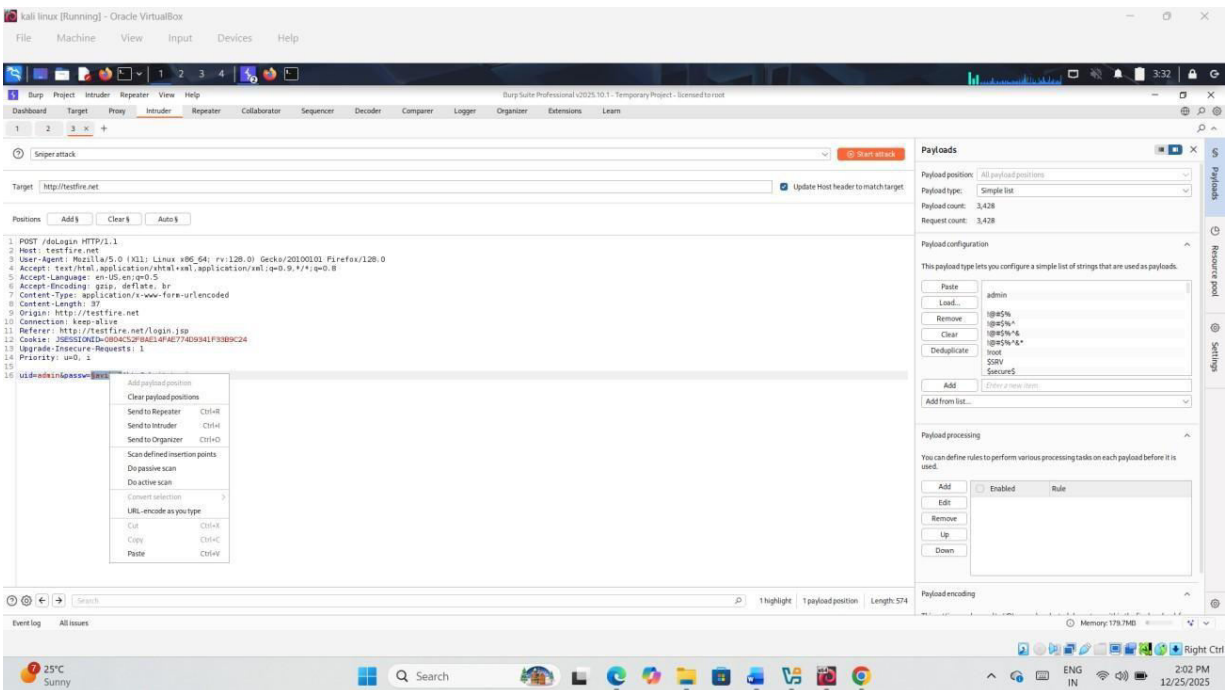
A **Sniper attack** is an **Intruder attack type in Burp Suite** where **one payload position is tested at a time** while all other parameters remain constant.



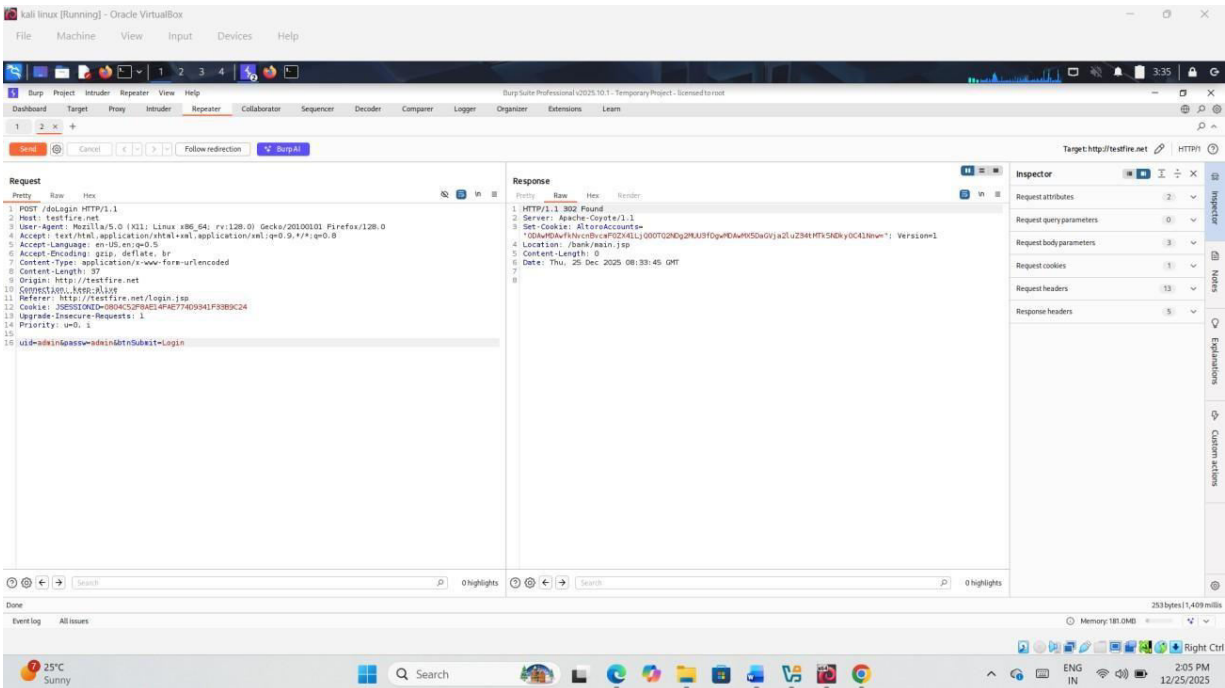
POC'S



POC'S

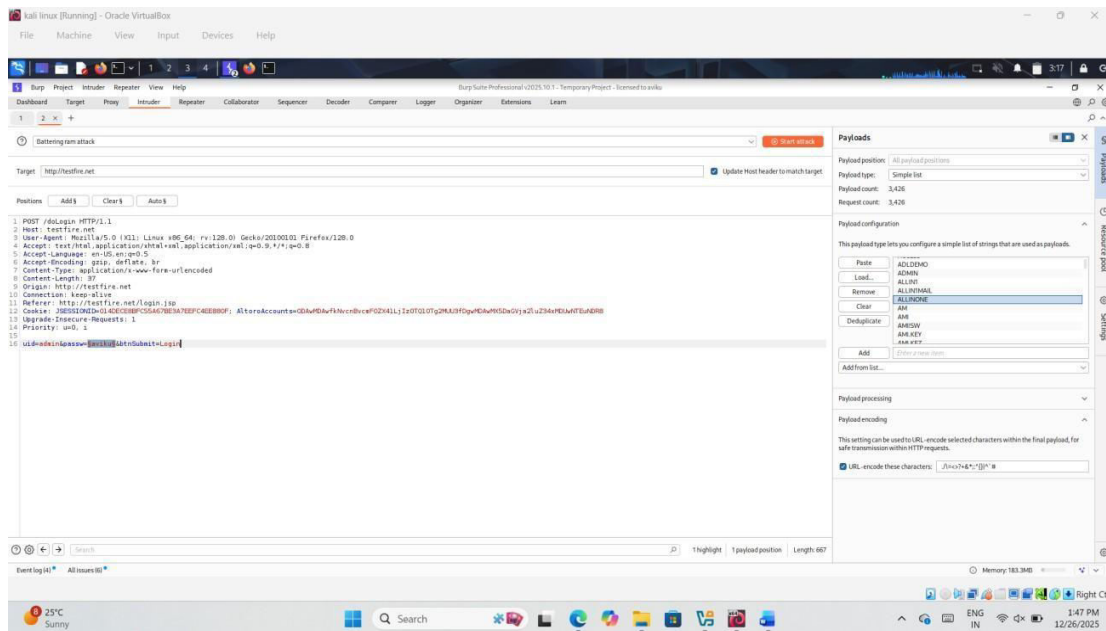


POC'S



Battering ram attack

A **Battering Ram attack** is an **Intruder attack type in Burp Suite** where the **same payload is injected into multiple positions simultaneously** in a single request.



POC'S

kali linux [Running] - Oracle VirtualBox

File Machine View Input Devices Help

Attack Save

4. Intruder attack of http://testfire.net

Results Positions

View filter: Showing all items

Apply capture filter

Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
14	ADMIN	302	352			236	
50	admin	302	308			236	
1067	admin	302	362			236	
3425	admin	302	354			236	
3426	admin	302	309			236	
838	Slings	302	304			145	
839	Slings	302	307			145	
841	Slayer	302	306			145	
845	Sowler	302	405			145	
853	Spanish	302	309			145	
856	Special	302	304			145	
859	USDR	302	298			145	
974	Vette	302	306			145	
1003	Whales	302	309			145	
1573	dallas	302	292			145	
1574	dan	302	304			145	

Request Response

1 HTTP/1.1 302 Found

2 Server: Apache/2.4.18 (Ubuntu)

3 Set-Cookie: ALTORAccount=IDbWHDwFhNcnbVcf0ZK4L3IzOT0LOtq2M3U9FgWpWfH5DwGj3aZUz34HdMHTbLADRB

4 Location: /bank/main.jsp

5 Content-Length: 0

6 Date: Fri, 26 Dec 2025 08:10:26 GMT

7

8

Altoro Mutual

MY ACCOUNT PERSONAL SMALL BUSINESS INSIDE ALTORO MUTUAL

I WANT TO...

- View Account Summary
- View Recent Transactions
- Transfer Funds
- Search News Articles
- Customize Site Language

ADMINISTRATION

- Exit Users

Hello Admin User

Welcome to Altoro Mutual Online.

View Account Details: 800000 Corporate GO

Congratulations!

You have been pre-approved for an Altoro Gold Visa with a credit limit of \$10000!

Click [Here](#) to apply.

Privacy Policy | Security Statement | Server Status Check | BEST AD | © 2025 Altoro Mutual, Inc.

This web application is open source! Get your copy from [Github](#) and take advantage of advanced features

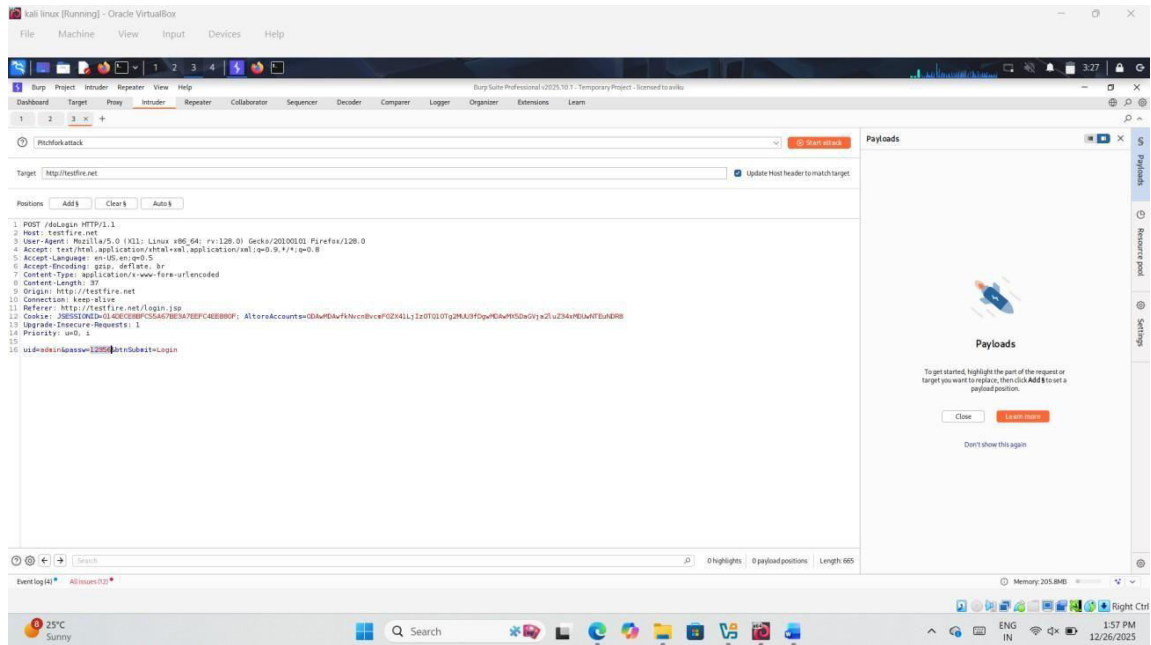
The Altoro website is published by HCL Technologies, Ltd. for the sole purpose of demonstrating the effectiveness of HCL products in detecting web application vulnerabilities and website defects. This site is not a real banking site. Similarities, if any, to third party products and/or websites are purely coincidental. This site is provided "as is" without warranty of any kind, either express or implied. HCL does not assume any risk in relation to your use of this website. For more information, please go to <https://www.hcl-software.com/appsec>.

Copyright © 2006, 2017, IBM Corporation. All rights reserved. Copyright © 2017, 2025, HCL Technologies, Ltd. All rights reserved.

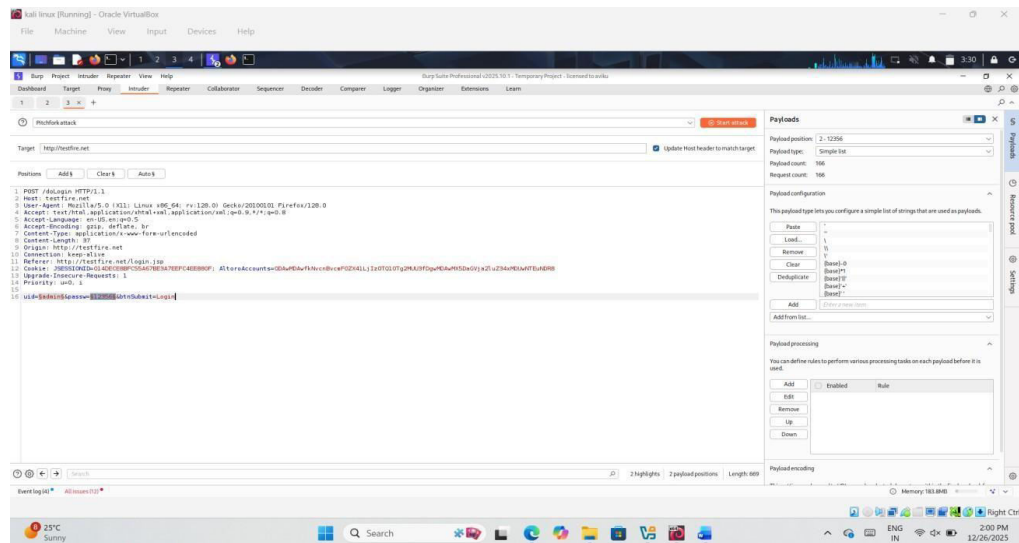
POC'S

Pitchfork attack

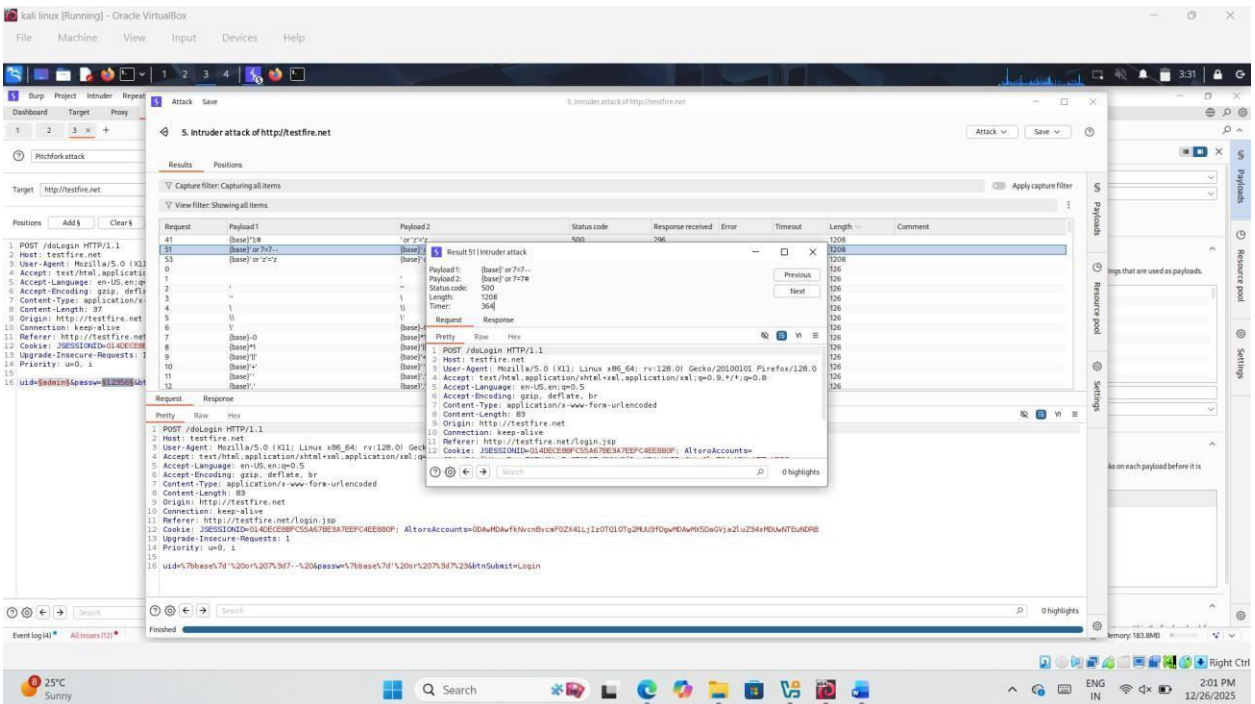
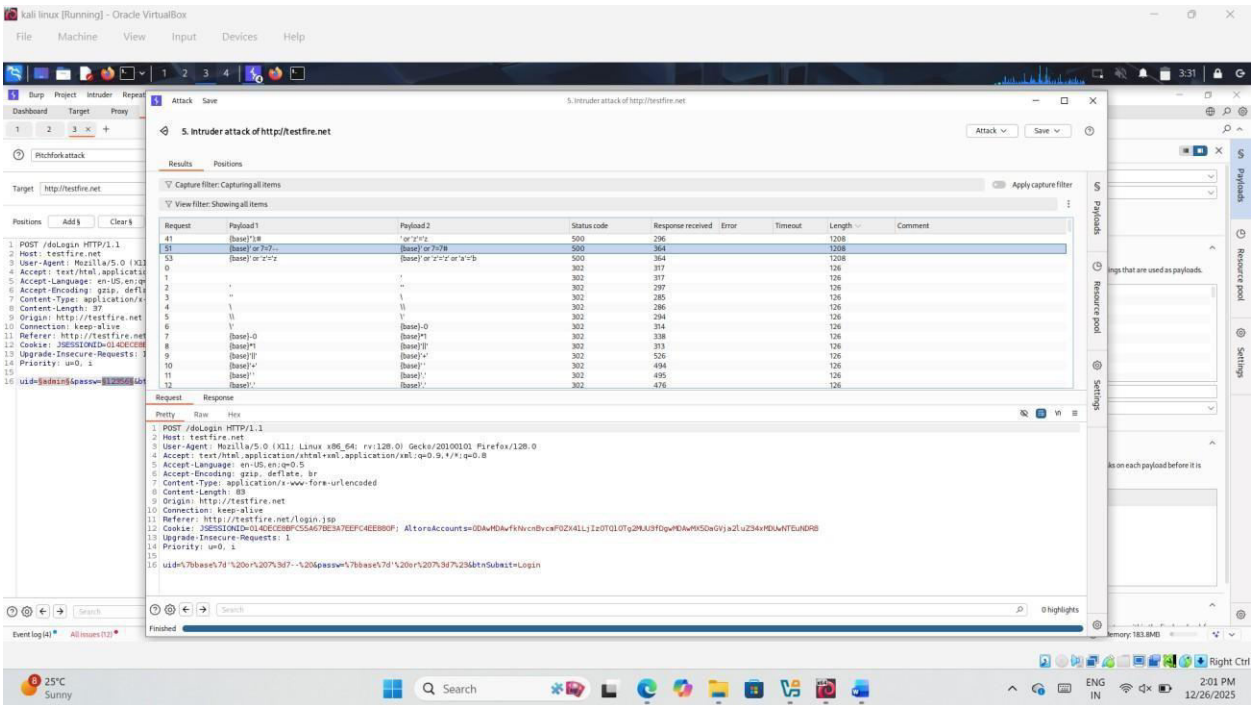
A Pitchfork attack is an Intruder attack type in Burp Suite where multiple payload sets are used in parallel, and each payload from one set is matched with the payload at the same position in the other set(s).



Setting payload on the uid & password



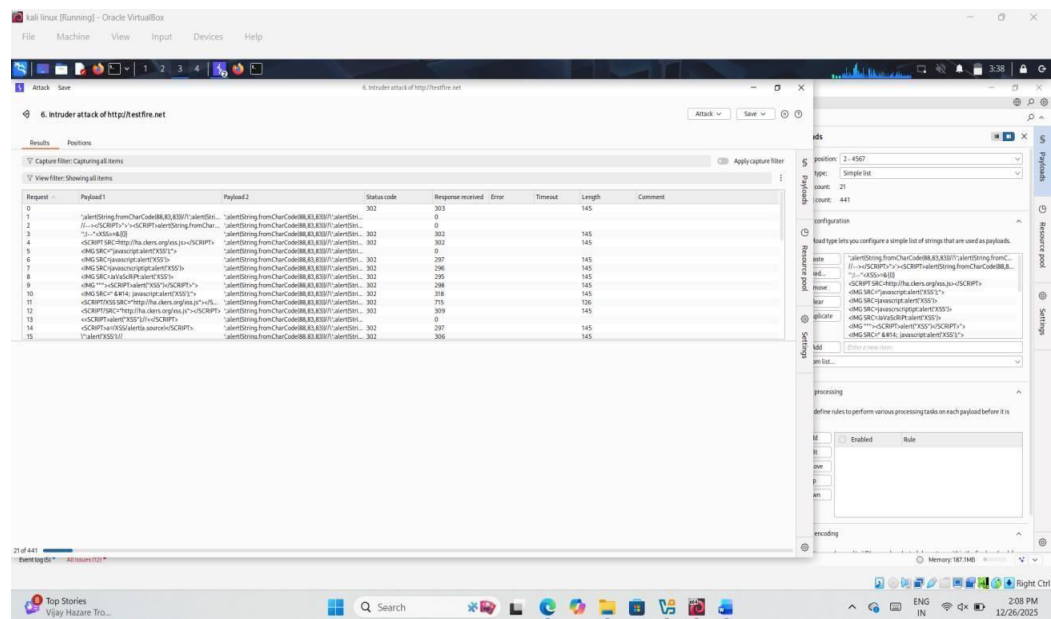
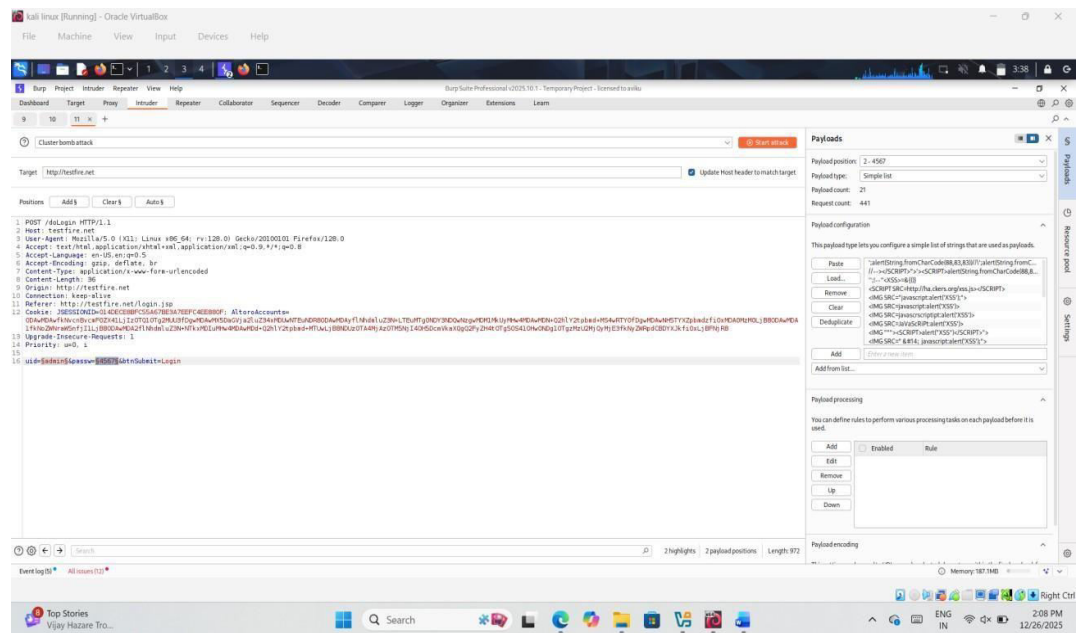
POC'S



POC'S

Cluster bomb attack

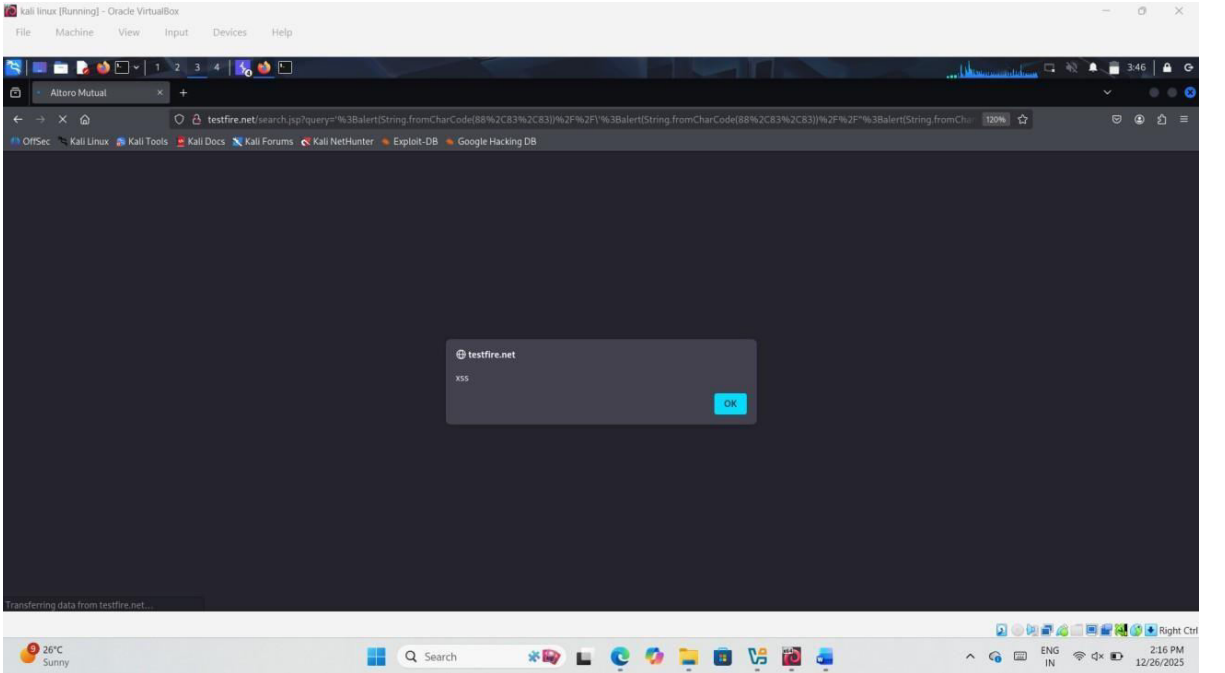
A Cluster Bomb attack is an Intruder attack type in Burp Suite where multiple payload sets are tested in all possible combinations.



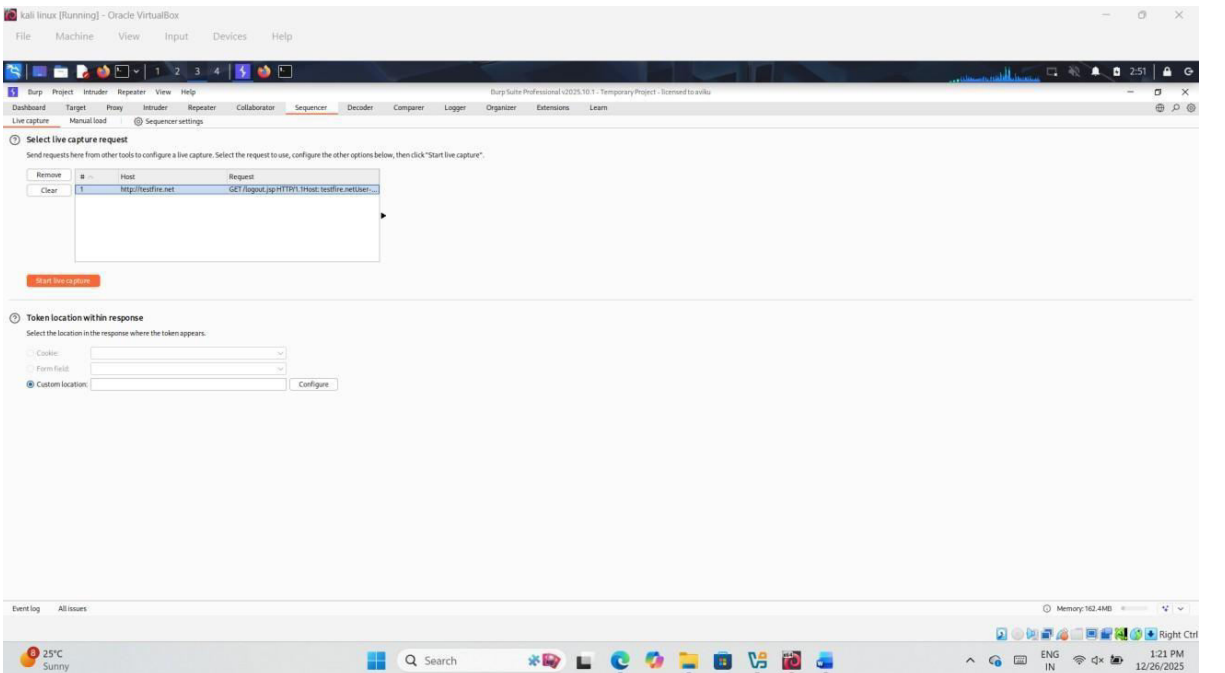
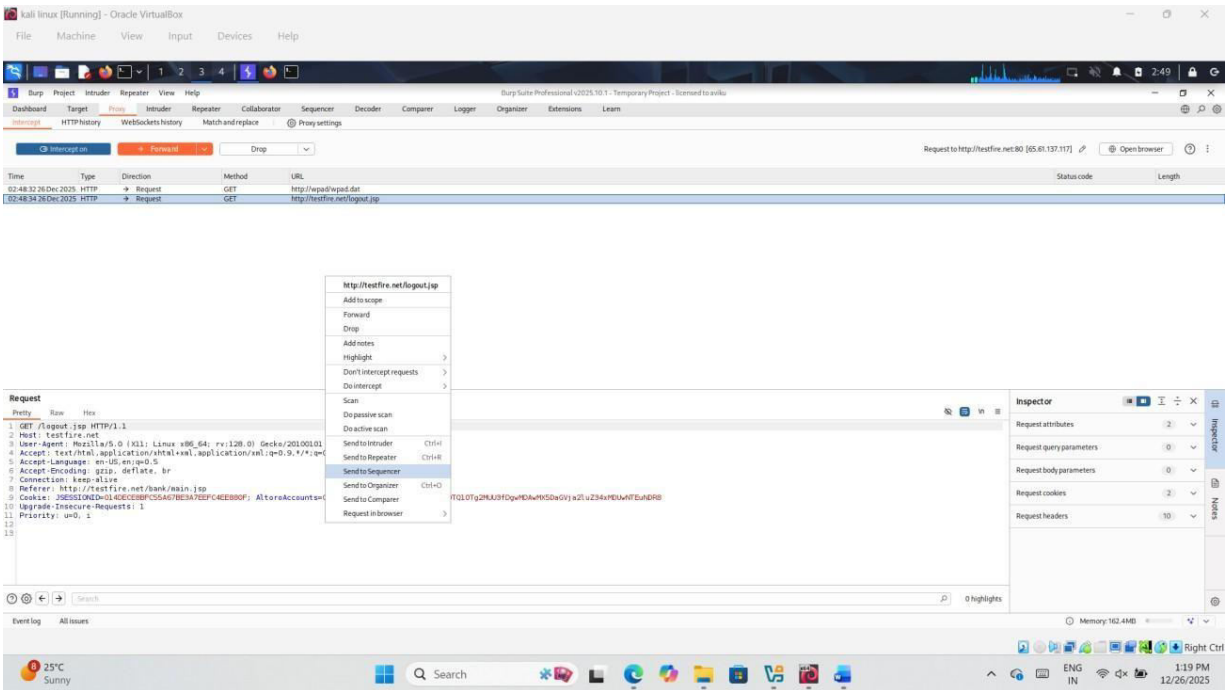
POC'S

The screenshot shows a Kali Linux virtual machine running Burp Suite. The main window displays the 'Results' tab for an attack on http://testfire.net. A list of requests and responses is shown, with a detailed view of a POST request to /doLogin HTTP/1.1. The response shows a 302 status code and a redirect to /login.jsp. The Burp Suite interface includes a 'Payloads' list on the right, showing various payloads used in the attack. The bottom status bar indicates the system is running on a virtual machine with a date of 12/26/2025.

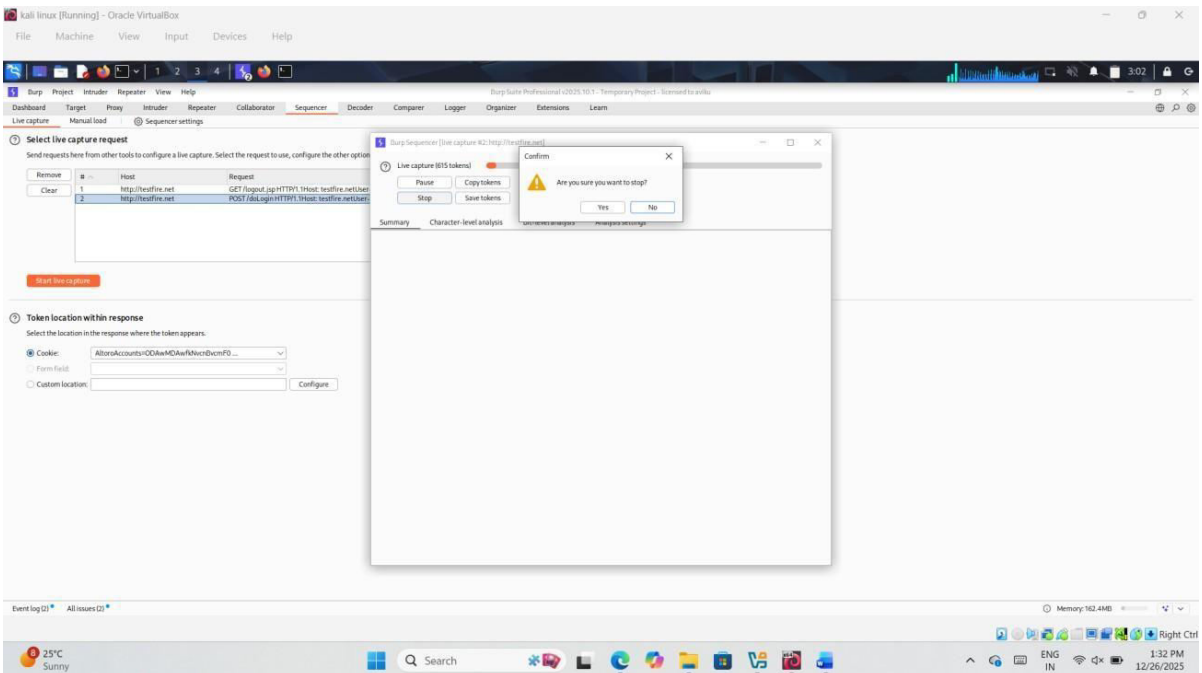
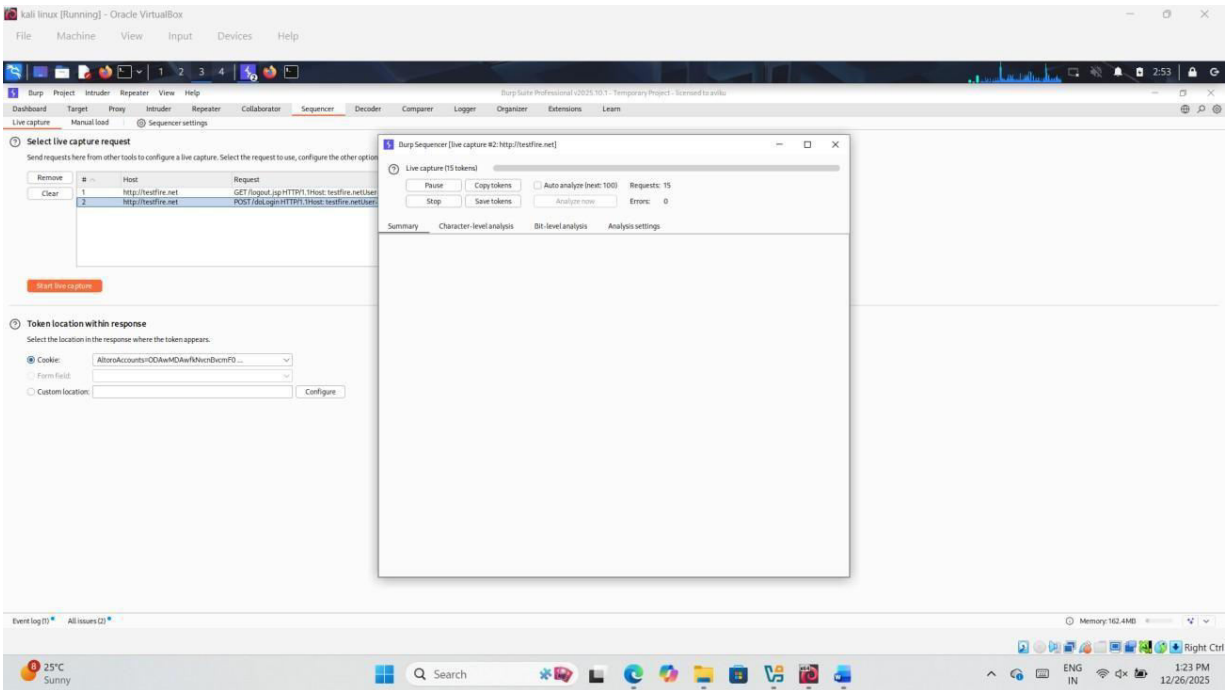
POC'S



POC'S



POC'S



POC'S

